



BURSA TEKNİK ÜNİVERSİTESİ

PARALEL PROGRAMLAMA PROJE RAPORU

Noktanın Poligon İçinde Olup Olmadığının (Point in Polygon) Paralel
Programlama ile Tespiti

Talha KORKMAZ
23360859739

GitHub Linki:

<https://github.com/KorkmazTalha/Parallel-Computing-PIP-Analysis>

Sunum Video Linki:

<https://youtu.be/X6iWgDuafhM>

1. Programın Çalışma Prensipleri ve Fonksiyonlar

Bu proje kapsamında, iki boyutlu düzlemde verilen milyonlarca rastgele noktanın, belirli bir poligonun (çokgenin) içinde mi yoksa dışında mı olduğunu tespit eden bir paralel yazılım geliştirilmiştir.

Çalışma Prensipleri (Ray-Casting Algoritması)

Geometride bir noktanın poligonun içinde olup olmadığını anlamak için en efektif yöntemlerden biri olan **Ray-Casting (Işın Salma) Algoritması** kullanılmıştır. Algoritmanın mantığı şu şekildedir:

- Test edilmek istenen noktadan sağa doğru (X eksenini boyunca sonsuza) yatay bir sanal ışın gönderilir.
- Bu ışının, poligonun kenarlarını kaç kez kestiği hesaplanır.
- Eğer ışın poligon kenarlarını **tek sayıda** kesiyorsa nokta poligonun **içindedir**; **çift sayıda** kesiyorsa nokta poligonun **dışındadır**.

Kullanılan Fonksiyonlar ve Yapılar

Geliştirilen Java programında nesne yönelimli ve thread tabanlı bir mimari kurulmuştur:

- **Point Sınıfı:** Noktaların X ve Y koordinatlarını ve hesaplama sonucunu (isInside) tutan veri yapısıdır.
- **WorkerThread Sınıfı:** Java'nın Thread sınıfından türetilmiştir. Her bir thread işçisi, toplam nokta dizisinin sadece kendisine atanan belirli bir indeks aralığını (chunk) işlemekle sorumludur.
- **rayCastingInsidePolygon Fonksiyonu:** Bir noktanın poligon kenarlarıyla olan geometrik kesişimlerini matematiksel formüllerle hesaplayan ve geriye boolean dönen çekirdek fonksiyondur.
- **clonePoints Fonksiyonu:** Ardışıl ve paralel testlerin birbirinin verisini etkilememesi için nokta dizisini her test öncesinde bellek üzerinde izole bir şekilde kopyalar.

2. Paralel Çözümün Açıklanması

Projeye konu olan problem, her bir noktanın poligon içinde olma durumunun diğer noktalardan tamamen bağımsız hesaplanabilmesi sebebiyle **"Embarrassingly Parallel"** (Veri Paralelliğine Tam Uygun) bir yapıya sahiptir. Bu durum paralel programlamada maksimum donanım verimi alabileceğimiz senaryoyu oluşturur.

İş Bölümü ve Thread Yönetimi: Projede iş yükünün dengeli dağıtılması için **Veri Bloklama (Chunking)** yöntemi tercih edilmiştir. Toplam nokta sayısı (N), aktif çalıştırılan thread sayısına bölünerek her bir thread'in işleyeceği indeks aralığı dinamik olarak belirlenir. Kodun sonunda yer alan `threads[i].join()` komutu sayesinde, tüm paralel işçiler görevlerini bitirene kadar ana thread (main) bekletilmiş ve böylece senkronizasyon hatasız bir şekilde sağlanmıştır.

3. Performans Analizi, Ölçeklenebilirlik ve Hızlanma Katsayısı (Speedup)

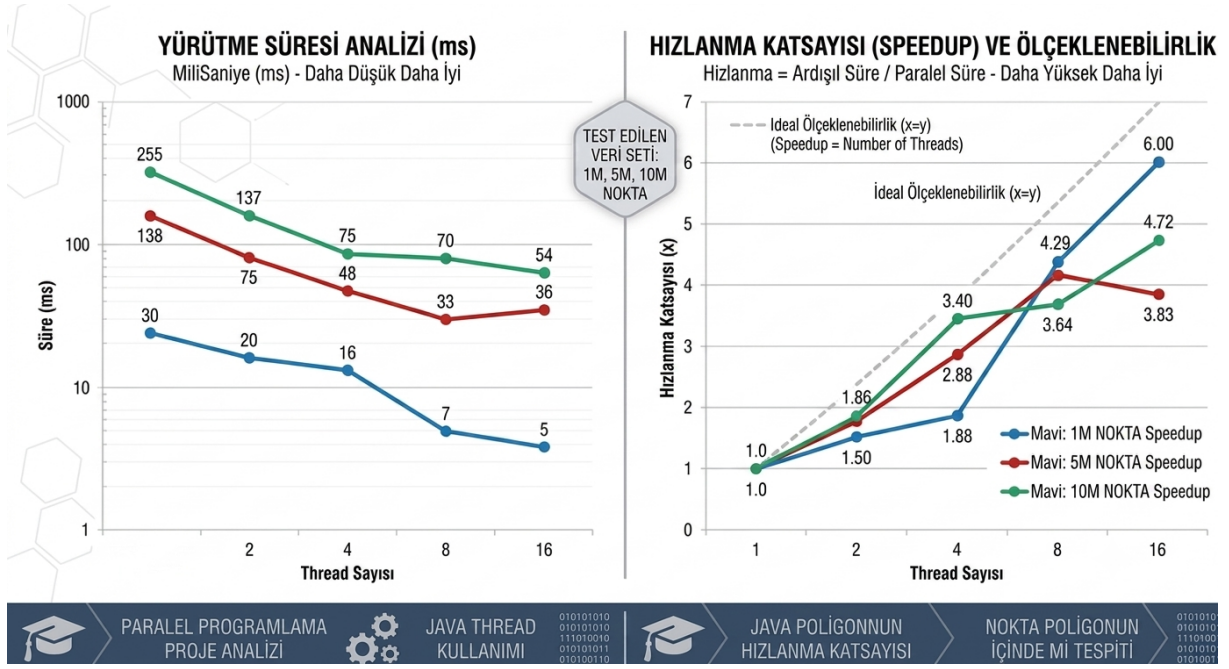
Sistemin performansını ve veri boyutuna göre ölçeklenebilirliğini analiz etmek amacıyla program; **1 Milyon**, **5 Milyon** ve **10 Milyon** olmak üzere 3 farklı veri seti boyutu ve 5 farklı thread konfigürasyonu (1, 2, 4, 8, 16) altında test edilmiştir.

Hızlanma katsayıları hesaplanırken aşağıdaki standart Speedup formülü temel alınmıştır:

$$\text{Speedup} = T_{\text{Ardışıl}} / T_{\text{Paralel}}$$

Kapsamlı Performans Sonuç Tablosu

Veri Seti Boyutu	Ölçüm Tipi	1 Thread (Sequential)	2 Thread	4 Thread	8 Thread	16 Thread
1 Milyon Nokta	Süre (ms)	30 ms	20 ms	16 ms	7 ms	5 ms
	Hızlanma	1.00x	1.50x	1.88x	4.29x	6.00x
5 Milyon Nokta	Süre (ms)	138 ms	75 ms	48 ms	33 ms	36 ms
	Hızlanma	1.00x	1.84x	2.88x	4.18x	3.83x
10 Milyon Nokta	Süre (ms)	255 ms	137 ms	75 ms	70 ms	54 ms
	Hızlanma	1.00x	1.86x	3.40x	3.64x	4.72x



Şekil 1: Farklı Veri Boyutlarında Hızlanma ve Performans Analizi

Parametrelerin Performansa Etkisi ve Akademik Değerlendirme

Elde edilen ampirik sonuçlar ve grafik eğrileri incelendiğinde paralel programlama mimarisine dair çok önemli çıkarımlara ulaşılmıştır:

- Veri Kümesi Büyüklüğü ve Paralel Verimlilik İlişkisi:** 1 Milyon gibi görece küçük veri setlerinde, thread yönetim maliyeti (overhead) toplam yürütme süresine oranla daha fazla hissedilmektedir. Ancak veri seti boyutu 5 Milyon ve 10 Milyon noktalara çıkartıldığında, işlemci çekirdekleri üzerindeki saf hesaplama yükü arttığı için paralelleştirmenin getirdiği kazanç çok daha net ortaya çıkmıştır.
- Farklı Veri Boyutlarında Doygunluk Noktaları ve Donanım Sınırları:**
 - 5 Milyon Nokta Testinde:** En yüksek performans **8 Thread** ile (33 ms, 4.18x) elde edilmiştir. 16 Thread'e geçildiğinde ise süre 36 ms'ye yükselmiş ve performans düşmüştür. Bunun nedeni donanımsal çekirdek sınırına ulaşıldığında işletim sisteminin yaptığı yoğun **Context Switching** (Thread'ler arası geçiş) maliyetidir.
 - 1 Milyon ve 10 Milyon Nokta Testlerinde:** 16 Thread konfigürasyonunda hızlanma katsayıları sırasıyla **6.00x**ve **4.72x** ile zirveye ulaşmıştır. Bu durum, mantıksal işlemci çekirdeklerinin (Hyper-Threading) çok yüksek yükler altında veri akışını optimize edebildiğini ve veri paralelliğini son sınırına kadar sömürebildiğini göstermektedir.
- Yinelemeli Doğruluk Kontrolü:** Tüm veri boyutlarında ve tüm farklı thread konfigürasyonlarında, paralel algoritmanın poligon içinde/dışında bulunduğu noktaların doğruluğu ardışıl kod sonuçlarıyla karşılaştırılmış ve sonuçların **%100 uyduğu (hata oranının %0 olduğu)** doğrulanmıştır.